

Barrett Viator DSE6111 - Preliminary Results (Resubmission)

Barrett Viator

2024-12-15

Client Attrition Preliminary Results

When assessing the results from the preliminary logistic regression model and from KNN models against the 0.841 accuracy level of the null model, it is apparent that further models will need to be explored to assess the risk of attrition based on past customer behaviors. The Logistic Regression model is performing well with an accuracy score of 0.9093 and an AUC of 0.9276. The KNN AUC value of 0.8585 suggests that the KNN model is performing well with the classification process of Attrition Flags which also pairs well with the accuracy value of 0.8735 for the confusion matrix.

Further processing of the data with subset selection and shrinkage methods applied has the advantage of providing ABC Corporation with better model accuracy as well as the ability to narrow down which features are most greatly impacting the predictors. Applying boosting, bagging, and Random Forest Methods as well as exploring XG-boost will also enhance the ability to improve upon the preliminary models to move forward with final recommendations more precisely and soundly.

Preparations, Data Cleaning, and Splitting

```
train_data$Attrition_Flag <- as.factor(train_data$Attrition_Flag)

# Balance the data using oversampling (default method of ovun.sample)
balanced_data <- ovun.sample(Attrition_Flag ~ ., data = train_data, method = "over", N = 5697)$data
```

Null Model

```
null_model <- balanced_data %>%
  count(Attrition_Flag) %>%
  top_n(1, n) %>%
  pull(Attrition_Flag)

null_predictions <- tibble(Attrition_Flag = rep(null_model, nrow(test_data)))

null_model_accuracy <- mean(null_predictions$Attrition_Flag == test_data$Attrition_Flag)
cat("Test Accuracy for the Null Model: ", null_model_accuracy, "\n")

## Test Accuracy for the Null Model: 0.8424658
```

CV Sets and Recipes

```
cv_set <- vfold_cv(train_data2, strata = "Attrition_Flag", v = 10)

customer_attrition_recipe <- recipe(Attrition_Flag ~ ., data = train_data2) %>%
```

```

step_dummy(all_nominal_predictors()) %>%
step_center(all_predictors()) %>%
step_scale(all_predictors())

```

Logistic Regression

```

train_data$Attrition_Flag <- as.character(train_data$Attrition_Flag)
prepared_recipe <- prep(customer_attrition_recipe, training = train_data2)

processed_train_data <- bake(prepared_recipe, new_data = train_data2)

customer_log_reg_mod <-
  logistic_reg(penalty = tune(), mixture = 1) %>%
  set_engine("glmnet")

customer_log_reg_workflow <-
  workflow() %>%
  add_recipe(customer_attrition_recipe) %>%
  add_model(customer_log_reg_mod)

customer_log_reg_grid <- tibble(penalty = 10^seq(-4, -1, length.out = 30))

log_reg_tuning_results <- tune_grid(
  object = customer_log_reg_workflow,
  resamples = cv_set,
  grid = customer_log_reg_grid,
  metrics = metric_set(roc_auc)
)

```

```

top_log_models <- log_reg_tuning_results %>%
  show_best(metric = "roc_auc", n = 15) %>%
  arrange(penalty)

```

```
top_log_models
```

```

## # A tibble: 15 x 7
##   penalty .metric .estimator mean      n std_err .config
##   <dbl> <chr> <chr> <dbl> <int> <dbl> <chr>
## 1 0.0001 roc_auc binary 0.928 10 0.00439 Preprocessor1_Model01
## 2 0.000127 roc_auc binary 0.928 10 0.00439 Preprocessor1_Model02
## 3 0.000161 roc_auc binary 0.928 10 0.00439 Preprocessor1_Model03
## 4 0.000204 roc_auc binary 0.928 10 0.00439 Preprocessor1_Model04
## 5 0.000259 roc_auc binary 0.928 10 0.00440 Preprocessor1_Model05
## 6 0.000329 roc_auc binary 0.928 10 0.00438 Preprocessor1_Model06
## 7 0.000418 roc_auc binary 0.928 10 0.00440 Preprocessor1_Model07
## 8 0.000530 roc_auc binary 0.928 10 0.00443 Preprocessor1_Model08
## 9 0.000672 roc_auc binary 0.928 10 0.00441 Preprocessor1_Model09
## 10 0.000853 roc_auc binary 0.928 10 0.00440 Preprocessor1_Model10
## 11 0.00108 roc_auc binary 0.928 10 0.00443 Preprocessor1_Model11
## 12 0.00137 roc_auc binary 0.928 10 0.00446 Preprocessor1_Model12
## 13 0.00174 roc_auc binary 0.928 10 0.00448 Preprocessor1_Model13
## 14 0.00221 roc_auc binary 0.928 10 0.00455 Preprocessor1_Model14
## 15 0.00281 roc_auc binary 0.928 10 0.00464 Preprocessor1_Model15

```

```

log_result_metrics <- collect_metrics(log_reg_tuning_results)

log_roc_auc_results <- log_result_metrics %>%
  filter(.metric == "roc_auc")

print(log_roc_auc_results)

## # A tibble: 30 x 7
##   penalty .metric .estimator mean     n std_err .config
##   <dbl> <chr>   <chr>   <dbl> <int>   <dbl> <chr>
## 1 0.0001  roc_auc binary    0.928    10 0.00439 Preprocessor1_Model101
## 2 0.000127 roc_auc binary    0.928    10 0.00439 Preprocessor1_Model102
## 3 0.000161 roc_auc binary    0.928    10 0.00439 Preprocessor1_Model103
## 4 0.000204 roc_auc binary    0.928    10 0.00439 Preprocessor1_Model104
## 5 0.000259 roc_auc binary    0.928    10 0.00440 Preprocessor1_Model105
## 6 0.000329 roc_auc binary    0.928    10 0.00438 Preprocessor1_Model106
## 7 0.000418 roc_auc binary    0.928    10 0.00440 Preprocessor1_Model107
## 8 0.000530 roc_auc binary    0.928    10 0.00443 Preprocessor1_Model108
## 9 0.000672 roc_auc binary    0.928    10 0.00441 Preprocessor1_Model109
## 10 0.000853 roc_auc binary    0.928    10 0.00440 Preprocessor1_Model110
## # i 20 more rows

best_log_auc_model <- log_roc_auc_results %>%
  arrange(desc(mean))

head(best_log_auc_model)

## # A tibble: 6 x 7
##   penalty .metric .estimator mean     n std_err .config
##   <dbl> <chr>   <chr>   <dbl> <int>   <dbl> <chr>
## 1 0.00108  roc_auc binary    0.928    10 0.00443 Preprocessor1_Model111
## 2 0.000853 roc_auc binary    0.928    10 0.00440 Preprocessor1_Model110
## 3 0.00137  roc_auc binary    0.928    10 0.00446 Preprocessor1_Model112
## 4 0.000672 roc_auc binary    0.928    10 0.00441 Preprocessor1_Model109
## 5 0.000530 roc_auc binary    0.928    10 0.00443 Preprocessor1_Model108
## 6 0.00174  roc_auc binary    0.928    10 0.00448 Preprocessor1_Model113

train_data2$Attrition_Flag <- as.factor(train_data2$Attrition_Flag)

log_reg_spec <- logistic_reg() %>%
  set_engine("glm") %>%
  set_mode("classification")

log_reg_fit <- log_reg_spec %>%
  fit(Attrition_Flag ~ ., data = train_data2)

log_reg_pred_prob <- predict(log_reg_fit, test_data, type = "raw")
log_reg_pred_class <- ifelse(log_reg_pred_prob > 0.5, 1, 0)

log_reg_conf_matrix <- table(Predicted = log_reg_pred_class, Actual = test_data$Attrition_Flag)

print(log_reg_conf_matrix)

##           Actual

```

```

## Predicted Attrited Customer Existing Customer
##           0           196           88
##           1           103          1511

log_reg_accuracy <- sum(diag(log_reg_conf_matrix)) / sum(log_reg_conf_matrix)
print(paste("Accuracy: ", round(log_reg_accuracy, 3)))

## [1] "Accuracy:  0.899"

log_roc_curve <- roc(test_data$Attrition_Flag, log_reg_pred_prob)

## Setting levels: control = Attrited Customer, case = Existing Customer
## Setting direction: controls < cases

log_auc <- auc(log_roc_curve)

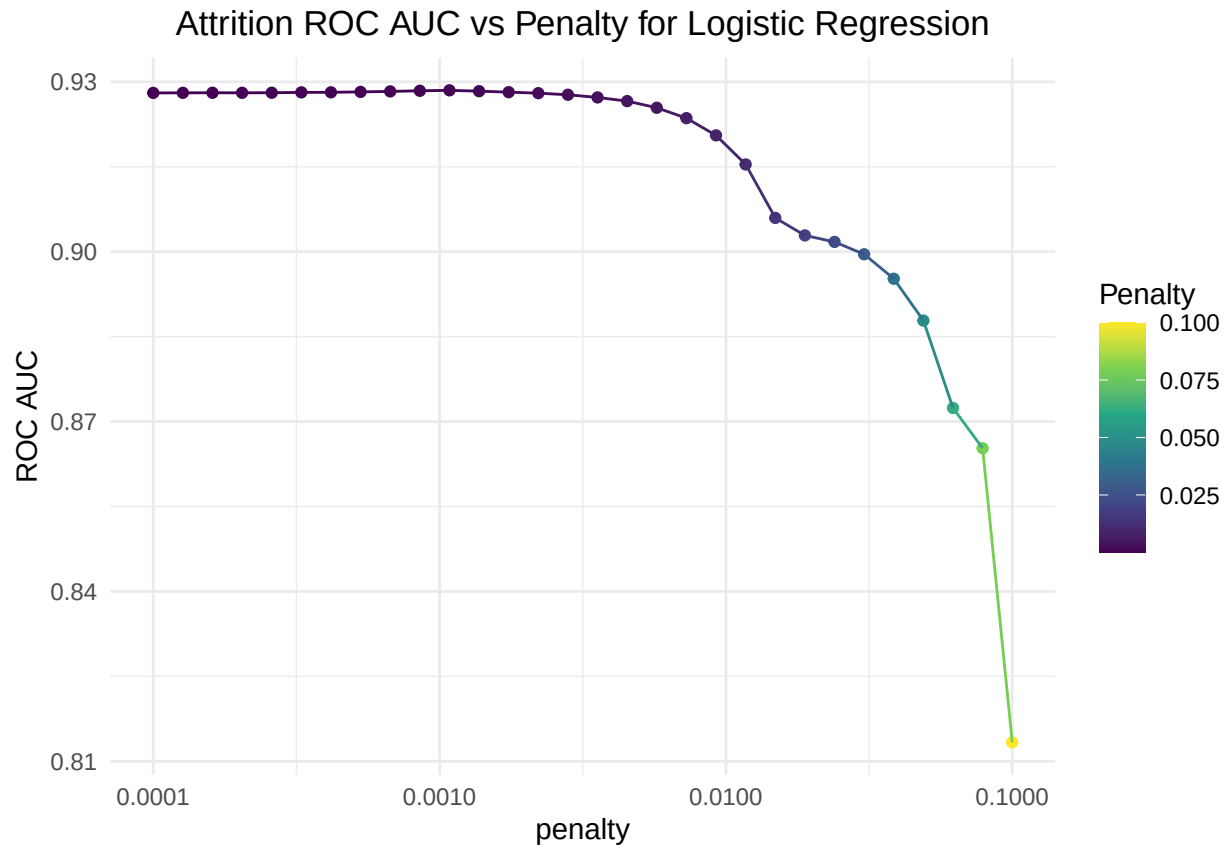
print(paste("AUC:", log_auc))

## [1] "AUC: 0.924413460754108"

log_reg_roc_plot <-
  log_reg_tuning_results %>%
  collect_metrics() %>%
  filter(.metric == "roc_auc") %>%
  ggplot(aes(x = penalty, y = mean, color = penalty)) + #
  geom_point() +
  geom_line() +
  ylab("ROC AUC") +
  scale_x_log10(labels = label_number()) +
  scale_color_continuous(type = "viridis", name = "Penalty") +
  ggtitle("Attrition ROC AUC vs Penalty for Logistic Regression") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5))

log_reg_roc_plot

```



```

train_data2$Attrition_Flag <- as.character(train_data2$Attrition_Flag)
log_best_model <- log_reg_tuning_results %>%
  select_best(metric = "roc_auc")

log_final_model <- customer_log_reg_workflow %>%
  finalize_workflow(log_best_model) %>%
  fit(data = train_data2)

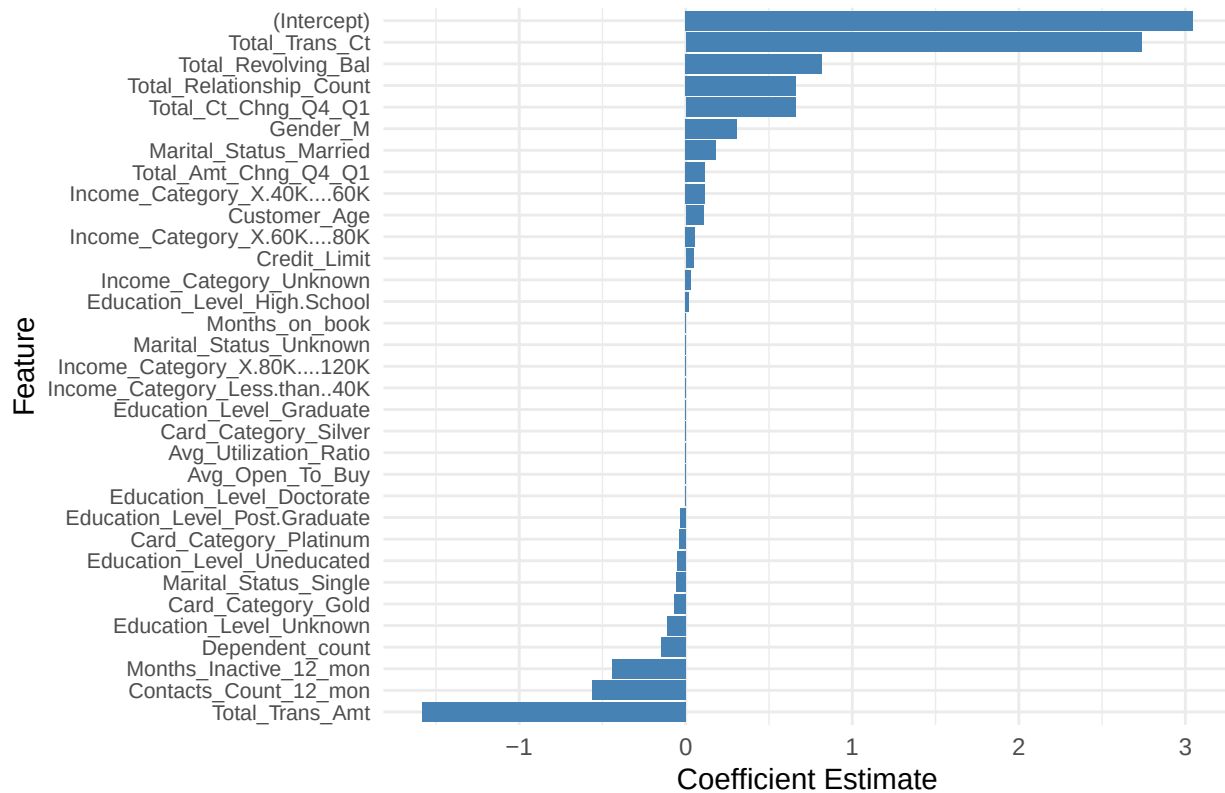
log_final_fit <- log_final_model %>%
  extract_fit_parsnip()

coefficients <- tidy(log_final_fit)

ggplot(coefficients, aes(x = reorder(term, estimate), y = estimate)) +
  geom_bar(stat = "identity", fill = "steelblue") +
  coord_flip() +
  labs(title = "Feature Importance (Logistic Regression)",
       x = "Feature",
       y = "Coefficient Estimate") +
  theme_minimal() +
  theme(axis.text.y = element_text(size = 8))

```

Feature Importance (Logistic Regression)



KNN

```
train_data$Attrition_Flag <- as.factor(train_data$Attrition_Flag)

customer_attrition_recipe4 <- recipe(Attrition_Flag ~
  Customer_Age +
  Gender +
  Dependent_count +
  Education_Level +
  Marital_Status +
  Income_Category +
  Card_Category +
  Months_on_book +
  Total_Relationship_Count +
  Months_Inactive_12_mon +
  Contacts_Count_12_mon +
  Credit_Limit +
  Total_Revolving_Bal +
  Avg_Open_To_Buy +
  Total_Amt_Chng_Q4_Q1 +
  Total_Trans_Amt +
  Total_Trans_Ct +
  Total_Ct_Chng_Q4_Q1 +
  Avg_Utilization_Ratio,
  data = train_data3)
```

```

customer_attrition_recipe4 <- customer_attrition_recipe4 %>%
  step_dummy(all_nominal(), -all_outcomes()) %>%
  step_scale(all_numeric(), -all_outcomes()) %>%
  step_center(all_numeric(), -all_outcomes()) %>%
  step_impute_median(all_numeric(), -all_outcomes())

train_data$Attrition_Flag <- as.factor(train_data$Attrition_Flag)
prepared_recipe4 <- prep(customer_attrition_recipe4, training = train_data3)

processed_train_data4 <- bake(prepared_recipe4, new_data = train_data)

cv_set4 <- vfold_cv(train_data3, v = 10, strata = Attrition_Flag)

knn_model <- nearest_neighbor(neighbors = tune()) %>%
  set_engine("kkn") %>%
  set_mode("classification")

knn_workflow <- workflow() %>%
  add_recipe(customer_attrition_recipe4) %>%
  add_model(knn_model)

knn_grid <- tibble(neighbors = seq(3, 15, by = 2))

knn_res <- knn_workflow %>%
  tune_grid(
    resamples = cv_set4,
    grid = knn_grid,
    control = control_grid(save_pred = TRUE),
    metrics = metric_set(roc_auc))

best_knn <- knn_res %>%
  select_best(metric = "roc_auc")

final_knn_workflow <- knn_workflow %>%
  finalize_workflow(best_knn)

final_knn_fit <- final_knn_workflow %>%
  fit(data = train_data3)

training_set$Attrition_Flag <- as.character(training_set$Attrition_Flag)
prepared_recipe3 <- prep(customer_attrition_recipe4, training = train_data3)
processed_train_data3 <- bake(prepared_recipe3, new_data = train_data3)

test_baked <- bake(prepared_recipe3, new_data = test_data2)

knn_test_pred <- predict(final_knn_fit, new_data = test_data2, type = "prob")

test_results_knn <- test_data2 %>%
  dplyr::select(Attrition_Flag) %>%
  bind_cols(knn_test_pred)

test_results_knn <- test_results_knn %>%
  rename(pred_Attrited_Customer = `.pred_Attrited Customer`)

```

```

test_results_knn$pred_class <- ifelse(test_results_knn$pred_Attrited_Customer > 0.5,
                                     "Attrited Customer",
                                     "Existing Customer")

test_results_knn$pred_class <- factor(test_results_knn$pred_class, levels = c("Existing Customer", "Attrited Customer"))
test_results_knn$Attrition_Flag <- factor(test_results_knn$Attrition_Flag, levels = c("Existing Customer", "Attrited Customer"))
test_results_knn$pred_class <- factor(test_results_knn$pred_class, levels = c("Existing Customer", "Attrited Customer"))

accuracy_knn <- accuracy_vec(truth = test_results_knn$Attrition_Flag, estimate = test_results_knn$pred_Attrited_Customer)

roc_auc_knn <- roc_auc_vec(truth = test_results_knn$Attrition_Flag,
                          estimate = test_results_knn$pred_Attrited_Customer, event_level = "second")

print(paste("ROC AUC:", roc_auc_knn))

## [1] "ROC AUC: 0.828330206378987"

print(paste("Accuracy:", accuracy_knn))

## [1] "Accuracy: 0.868282402528978"

roc_curve_knn <- roc_curve(
  test_results_knn,
  truth = Attrition_Flag,
  pred_Attrited_Customer,
  event_level = "second"
)

roc_auc_knn <- roc_auc(test_results_knn, truth = Attrition_Flag, pred_Attrited_Customer)

autoplot(roc_curve_knn) +
  ggtitle(paste("ROC Curve for KNN Model", sep = "")) +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5))

```